Harrison Lloyd
CSCI 485
Prof. Shen
3/05/26

Homework 3

Task 1:

For this task, I found an online image called baboon.jpg. This image was already 256x256 and in the grayscale format. Since the dimensions can be divided by 8 without remainder, I didn't need to crop or resize them. As a precaution, I still used the line .convert('L') to convert the image to grayscale.

Task 2:

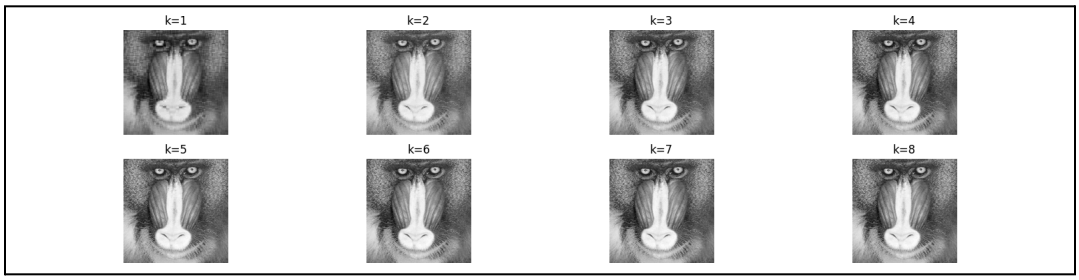
As wanted, I implemented a function called compress_block() to handle the compression process. It breaks the image into 8×8 blocks and loops through each one. For every block, it runs SVD to get the U matrix, singular values, and V^T matrix, then keeps only the top k parts from each. Those reduced matrices get multiplied back together to rebuild the compressed block, and then all the blocks are put back together into the final image.

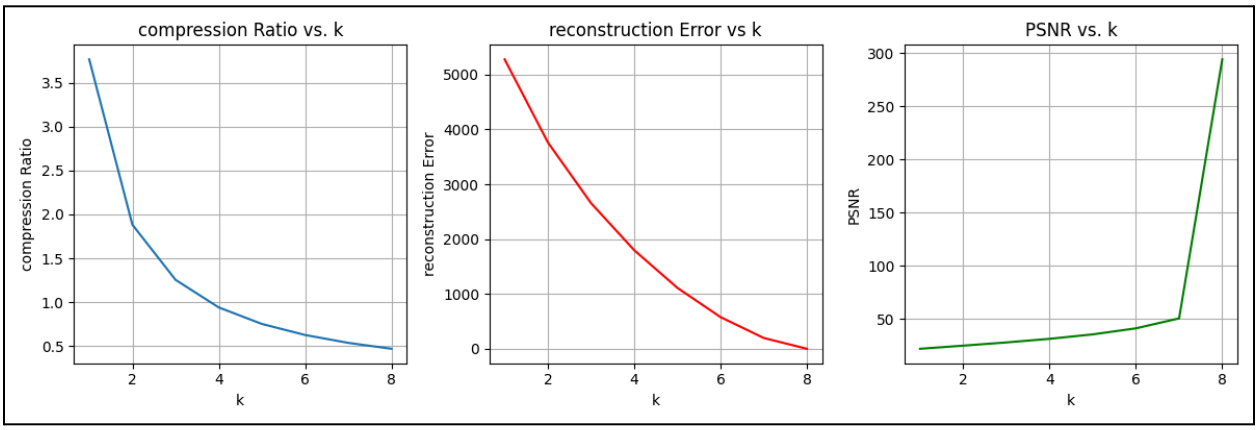
Task 3:

For this task, I ran the compression for k values 1 through 8. The compression ratio shows how much smaller the data gets, which was calculated as 64/17k, as per the formula on the assignment. The reconstruction error tells you how different the compressed image is from the original. I also threw in PSNR calculations to measure quality, which basically converts the mean squared error into decibels.

Task 4:

For this task, I created plots to show how the metrics change with different k values. The first plot shows compression ratio vs k, the second shows reconstruction error vs k, and the third shows PSNR vs k. I also made a figure showing all 8 compressed images side by side so you can compare the quality at different compression levels more easily (which was also part of task 3).


k=1
k=2
k=3
k=4
k=5
k=6
k=7
k=8


compression Ratio vs. k
compression Ratio
k
reconstruction Error vs k
reconstruction Error
k
PSNR vs. k
PSNR
k

ChatGPT Citation:

I had asked Chat how to compute the reconstruction error, and it told me this.

```
To compute reconstruction error:

You take the original image and subtract it by the reconstructed image, then calculate the Frobenius norm of that difference.

Code example:
difference = original_img - reconstructed_img
error = np.linalg.norm(difference)

The result is a single number representing the total error between the two images.
```

I had asked Chat, my images are tiny, how do I fix this? Chat told me to use figsize=() inside of the plt.figure()

I had asked Chat, what should we use to measure how different the reconstructed image is from the original in terms of pixel values? Chat told me this:

```
You should use mean squared error (MSE).

It takes the difference between each pixel in the original and reconstructed images, squares those differences, and then averages them all
together. This gives you a single number that represents the overall pixel-level difference between the two images. The squaring makes larger
errors more significant, so big differences get penalized more than small ones.
```

Lastly, I asked Chat, what's the formula to compute PSNR for images? It told me to use this formula below, and so I implemented the code myself based on what it told me.

```
The formula to compute PSNR for images is:

You take the maximum pixel value (255 for grayscale) and divide it by the square root of the mean squared error. Then you take the log base 10 of
that result and multiply by 20 to convert it to decibels.

So it's: 20 × log10(max_pixel / sqrt(MSE))

The 20 multiplier converts the ratio into decibel units, and the log10 compresses the range to make the values more interpretable. Higher PSNR
means better image quality.
```